Experiment 6 – Apply informed search on given problem

Learning Objective: To understand and implement informed search algorithms such as Greedy Best-First Search and A* Search for solving problems, and to analyse how heuristic functions improve search efficiency, optimality, and decision-making in traversing state spaces using Python.

Tools: Visual Code Studio

Theory: -

In Artificial Intelligence, informed search algorithms are advanced problem-solving techniques that utilize additional knowledge, known as heuristics, to guide the search process more efficiently toward the goal state. Unlike uninformed search methods, informed search algorithms make intelligent decisions by estimating how close a given state is to the goal, thereby reducing unnecessary exploration and improving performance.

This experiment focuses on two widely used informed search techniques: Greedy Best-First Search and A* (A-Star) Search. Both algorithms use heuristic functions but differ in how they evaluate and expand nodes.

1. Heuristic Function (h(n))

A heuristic function is an estimate of the cost required to reach the goal from a given node. It plays a crucial role in informed search by guiding the algorithm toward the most promising path. A good heuristic should be admissible (never overestimates the true cost) and consistent to ensure optimal results, especially in A* search.

2. Greedy Best-First Search

Greedy Best-First Search selects the node that appears to be closest to the goal based solely on the heuristic value h(n). It uses a priority queue where nodes with the lowest heuristic value are expanded first.This approach is fast and efficient because it aggressively moves toward the goal. However, it does not consider the cost already incurred to reach a node, which means it may not always find the optimal solution. It is best suited for problems where speed is more important than accuracy.

3. A* (A-Star) Search Algorithm

A* Search is one of the most effective and widely used informed search algorithms. It combines both the actual cost from the start node (g(n)) and the estimated cost to the goal (h(n)) to evaluate nodes. The evaluation function is given by:

$f(n) = g(n) + h(n)$

Here,

- g(n) represents the cost from the start node to the current node
- h(n) represents the estimated cost from the current node to the goal

A* uses a priority queue to expand the node with the lowest f(n) value. If the heuristic is admissible and consistent, A* guarantees an optimal solution. It is widely used in pathfinding, navigation systems, robotics, and game development.

4. Comparison of Greedy Best-First Search and A*

- Greedy Best-First Search uses only h(n) and focuses on reaching the goal quickly but may produce suboptimal results.
- A* uses both g(n) and h(n), ensuring a balance between path cost and goal estimation, leading to optimal solutions.
- Greedy search is faster but less reliable, whereas A* is slightly slower but more accurate and complete.

5. Applications of Informed Search

Informed search algorithms are used in real-world applications such as GPS navigation systems, route planning, robotics pathfinding, game AI, and network routing. These algorithms significantly improve efficiency by reducing the number of nodes explored.

Overall, informed search techniques enhance problem-solving by incorporating domain knowledge through heuristics. Understanding these algorithms is essential for developing intelligent systems capable of making efficient and optimal decisions.

<u>Code & Output</u>: -

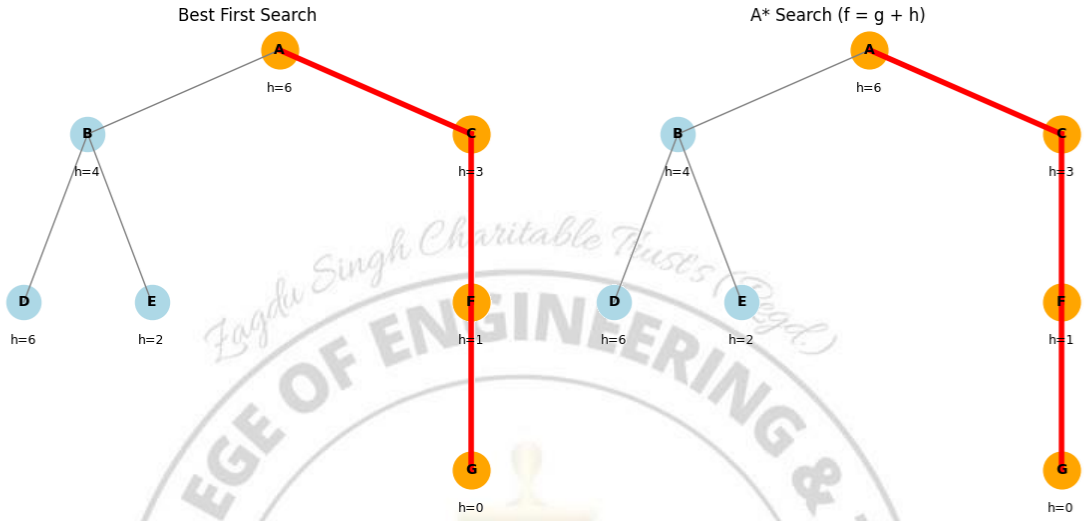
Input: -

```
import matplotlib.pyplot as plt
import heapq

# ---------- GRAPH ----------
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': ['G'],
    'G': []
}

cost_graph = {
    'A': [('B', 2), ('C', 1)],
    'B': [('D', 4), ('E', 1)],
    'C': [('F', 2)],
    'D': [],
    'E': [],
    'F': [('G', 1)],
    'G': []
}

heuristic = {
    'A': 6, 'B': 4, 'C': 3,
    'D': 6, 'E': 2, 'F': 1, 'G': 0
}

# ---------- POSITIONS ----------
pos = {
    'A': (0, 4),
    'B': (-3, 3),
    'C': (3, 3),
    'D': (-4, 1),
    'E': (-2, 1),
    'F': (3, 1),
    'G': (3, -1)
}

# ---------- DRAW FUNCTION ----------
def draw_graph(ax, title, path=None):

    # Draw edges (light)
    for u in graph:
        for v in graph[u]:
```

```
        x1, y1 = pos[u]
        x2, y2 = pos[v]
        ax.plot([x1, x2], [y1, y2], color='gray',
linewidth=1)

    # Highlight path
    if path:
        for i in range(len(path) - 1):
            x1, y1 = pos[path[i]]
            x2, y2 = pos[path[i + 1]]
            ax.plot([x1, x2], [y1, y2], color='red',
linewidth=4)

    # Draw nodes
    for n, (x, y) in pos.items():
        if path and n in path:
            ax.scatter(x, y, s=700, color='orange')  #
path nodes
        else:
            ax.scatter(x, y, s=600, color='lightblue')

        ax.text(x, y, n, ha='center', va='center',
fontweight='bold')
        ax.text(x, y - 0.5, f"h={heuristic[n]}",
ha='center', fontsize=9)

    ax.set_title(title)
    ax.axis('off')

# ---------- BEST FIRST SEARCH ----------
def best_first(start, goal):
    pq = [(heuristic[start], start, [start])]
    visited = set()

    while pq:
        _, node, path = heapq.heappop(pq)

        if node == goal:
            return path

        if node not in visited:
            visited.add(node)

            for n in graph[node]:
                heapq.heappush(pq, (heuristic[n], n,
path + [n]))

# ---------- A* SEARCH ----------
def a_star(start, goal):
    pq = [(heuristic[start], 0, start, [start])]  # f, g,
node, path
    visited = set()

    while pq:
        f, g, node, path = heapq.heappop(pq)

        if node == goal:
            return path, g

        if node not in visited:
            visited.add(node)

            for n, w in cost_graph[node]:
                g2 = g + w
                f2 = g2 + heuristic[n]
                heapq.heappush(pq, (f2, g2, n, path +
[n]))

# ---------- MAIN ----------

bf_path = best_first('A', 'G')
a_path, a_cost = a_star('A', 'G')

print("Best First Path:", bf_path)
print("A* Path:", a_path)
print("A* Cost:", a_cost)

# Create subplots (side-by-side)
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

draw_graph(axes[0], "Best First Search", bf_path)
draw_graph(axes[1], "A* Search (f = g + h)",
a_path)

plt.show()
```

Output: -

Best First Path: ['A', 'C', 'F', 'G']
A* Path: ['A', 'C', 'F', 'G']
A* Cost: 4



Fagdu Singh Charitable Trust's (Regd.)
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY
tcet
Estd. 2001
ISO 9001 : 2015 Certified
NBA and NAAC Accredited

Learning Outcomes: The student should have the ability to:

LO 6.1: Apply Greedy Best-First Search and A* Search algorithms to efficiently explore and solve state space problems.
LO 6.2: Utilize heuristic functions to guide the search process and improve performance.
LO 6.3: Analyse and compare the efficiency, optimality, and completeness of informed search algorithms.
LO 6.4: Implement informed search techniques for real-world problem-solving using Python.

Course Outcomes:

Upon completion of this experiment, students will be able to understand and apply informed search algorithms such as Greedy Best-First Search and A* Search, and evaluate their effectiveness in solving complex problems using heuristic-based approaches in Artificial Intelligence.

Conclusion:

Jagdu Singh Charitable Trust's (Regd.)
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY
Estd. 2001
ISO 9001 : 2015 Certified
NBA and NAAC Accredited

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				